

Лабораторная работа №8. Утилита Make

Цель: изучение системы сборки программ Make.

ЗАДАНИЕ

1. Изучите главу 9 теоретического курса «Утилита Make».
2. При необходимости установите программный пакет make.
3. Выполните на рабочем компьютере все практические примеры из данной главы.
4. Скачайте с сайта **www.busybox.net** исходный код программы Busybox и распакуйте его в подкаталог домашнего каталога.

```
guz@Guzovski:~$ mkdir busybox-src
guz@Guzovski:~$ cd busybox-src
guz@Guzovski:~/busybox-src$ wget https://busybox.net/downloads/busybox-1.36.1.tar.bz2
--2025-12-12 01:10:39-- https://busybox.net/downloads/busybox-1.36.1.tar.bz2
Resolving busybox.net (busybox.net)... 140.211.167.122
Connecting to busybox.net (busybox.net)|140.211.167.122|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2525473 (2.4M) [application/x-bzip2]
Saving to: 'busybox-1.36.1.tar.bz2'

busybox-1.36.1.tar. 100%[=====>] 2.41M 797KB/s in 3.1s

2025-12-12 01:10:43 (797 KB/s) - 'busybox-1.36.1.tar.bz2' saved [2525473/2525473]
```

Изучите файл `Makefile` из основного каталога проекта.

```

VERSION = 1
PATCHLEVEL = 36
SUBLEVEL = 1
EXTRAVERSION =
NAME = Unnamed

# *DOCUMENTATION*
# To see a list of typical targets execute "make help"
# More info can be located in ./README
# Comments in this file are targeted only to the developer, do not
# expect to learn how to build the kernel reading this file.

# Do not print "Entering directory ..."
MAKEFLAGS += --no-print-directory

# We are using a recursive build, so we need to do a little thinking
# to get the ordering right.
#
# Most importantly: sub-Makefiles should only ever modify files in
# their own directory. If in some directory we have a dependency on
# a file in another dir (which doesn't happen often, but it's often
# unavoidable when linking the built-in.o targets which finally
# turn into busybox), we will call a sub make in that other dir, and
# after that we are sure that everything which is in that other dir
# is now up to date.
#
# The only cases where we need to modify files which have global
# effects are thus separated out and done before the recursive
# descending is started. They are now explicitly listed as the
# prepare rule.

# To put more focus on warnings, be less verbose as default
# Use 'make V=1' to see the full commands

ifdef V
    ifeq ("$(origin V)", "command line")
        KBUILD_VERBOSE = $(V)
    endif
endif
ifndef KBUILD_VERBOSE
Makefile

```

Определите главную цель в файле и зависимости главной цели.

```
guz@Guzovski:~/busybox-src/busybox-1.36.1$ make
GEN      include/applets.h
GEN      include/usage.h
GEN      scripts/Kbuild
GEN      runit/Kbuild
GEN      runit/Config.in
GEN      networking/Kbuild
GEN      networking/Config.in
GEN      networking/libiproute/Kbuild
GEN      networking/udhcp/Kbuild
GEN      networking/udhcp/Config.in
GEN      console-tools/Kbuild
GEN      console-tools/Config.in
GEN      shell/Kbuild
GEN      shell/Config.in
GEN      e2fsprogs/Kbuild
GEN      e2fsprogs/Config.in
GEN      editors/Kbuild
```

Обратите внимание на переменные CROSS_COMPILE и ARCH в данном файле. Они предназначены для кросскомпиляции проекта. Исследуйте, каким образом они используются в файле.

```
guz@Guzovski:~/busybox-src/busybox-1.36.1$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-
/home/guz/busybox-src/busybox-1.36.1/scripts/gcc-version.sh: line 11: arm-linux-gnueabi-gcc: command not found
scripts/kconfig/conf -s Config.in
***
*** You have not yet configured busybox!
***
*** Please run some configurator (e.g. "make oldconfig" or
*** "make menuconfig" or "make defconfig").
***
make[2]: *** [/home/guz/busybox-src/busybox-1.36.1/scripts/kconfig/Makefile:40: silentoldconfig] Error 1
make[1]: *** [/home/guz/busybox-src/busybox-1.36.1/Makefile:444: silentoldconfig] Error 2
make: *** [Makefile:522: include/autoconf.h] Error 2
guz@Guzovski:~/busybox-src/busybox-1.36.1$
```

5. Разработайте свой собственный Makefile для задачи компиляции, выполняемой в предыдущей лабораторной работе. В файле должны присутствовать цели all, clean, install. Установка при выполнении цели install должна производиться в каталог, который определен в переменной INST_DIR, имеющей значение по умолчанию «/home/user/test_for_make/». Также в файле должны быть цели для сборки исполняемого файла, статической библиотеки и динамической библиотеки.

```
guz@Guzovski:~/busybox-src/busybox-1.36.1$ make
gcc -Wall -O2 -fPIC -o myprog main.o utils.o
ar rcs libmystatic.a utils.o
gcc -shared -o libmyshared.so utils.o
gcc -shared -o libmysnared.so utils.o
guz@Guzovski:~/busybox-src/busybox-1.36.1$ make clean
rm -f *.o myprog libmystatic.a libmyshared.so
guz@Guzovski:~/busybox-src/busybox-1.36.1$
```

```

# Компилятор и флаги
CC = gcc
CFLAGS = -Wall -O2

# Исходники и объекты
SRC = main.c utils.c
OBJ = $(SRC:.c=.o)

# Имена артефактов
TARGET = myprog
STATIC_LIB = libmylib.a
SHARED_LIB = libmylib.so

# Каталог установки (по умолчанию)
INST_DIR ?= /home/user/test_for_make/

# Главная цель
all: $(TARGET) $(STATIC_LIB) $(SHARED_LIB)

# Сборка исполняемого файла
$(TARGET): $(OBJ)
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJ)

# Сборка статической библиотеки
$(STATIC_LIB): $(OBJ)
    ar rcs $(STATIC_LIB) $(OBJ)

# Сборка динамической библиотеки
$(SHARED_LIB): $(OBJ)
    $(CC) -shared -o $(SHARED_LIB) $(OBJ)

# Очистка
clean:
    rm -f $(OBJ) $(TARGET) $(STATIC_LIB) $(SHARED_LIB)

guz@Guzovski:~/busybox-src/busybox-1.36.1$ nano Makefile
guz@Guzovski:~/busybox-src/busybox-1.36.1$ make
gcc -Wall -O2 -c -o main.o main.c
gcc -Wall -O2 -c -o utils.o utils.c
gcc -Wall -O2 -o myprog main.o utils.o
ar rcs libmylib.a utils.o
gcc -shared -o libmylib.so utils.o
guz@Guzovski:~/busybox-src/busybox-1.36.1$ make clean
rm -f main.o utils.o myprog libmylib.a libmylib.so
guz@Guzovski:~/busybox-src/busybox-1.36.1$ make install
gcc -Wall -O2 -c -o main.o main.c
gcc -Wall -O2 -c -o utils.o utils.c
gcc -Wall -O2 -o myprog main.o utils.o
ar rcs libmylib.a utils.o
gcc -shared -o libmylib.so utils.o
mkdir: cannot create directory '/home/user': Permission denied
make: *** [Makefile:38: install] Error 1

```

6. Доработайте созданный Makefile так, чтобы при компиляции можно было

использовать кросс-компилятор (используя переменные CROSS_COMPILE и ARCH).

```
# Компилятор и флаги
CROSS_COMPILE ?=
ARCH ?= $(shell uname -m)

CC = $(CROSS_COMPILE)gcc
CFLAGS = -Wall -O2

# Исходники и объекты
SRC = main.c utils.c
OBJ = $(SRC:.c=.o)

# Артефакты
TARGET = myprog
STATIC_LIB = libmylib.a
SHARED_LIB = libmylib.so

# Каталог установки (по умолчанию)
INST_DIR ?= /home/user/test_for_make/

# Главная цель
all: $(TARGET) $(STATIC_LIB) $(SHARED_LIB)

# Сборка исполняемого файла
$(TARGET): $(OBJ)
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJ)

# Сборка статической библиотеки
$(STATIC_LIB): utils.o
    ar rcs $(STATIC_LIB) utils.o

# Сборка динамической библиотеки
$(SHARED_LIB): utils.o
    $(CC) -shared -o $(SHARED_LIB) utils.o

# Очистка
clean:
    rm -f $(OBJ) $(TARGET) $(STATIC_LIB) $(SHARED_LIB)
```

[Read 44 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line

7. Запишите полученный Makefile и файлы, необходимые для компиляции, в подкаталог исходных кодов программы Busybox. Внесите необходимые изменения для того, чтобы разработанный вами файл вызывался при сборке программы Busybox (для этого нужно добавить в общий файл сборки дополнительную цель, которая запустит команду make в другом каталоге). Выполните сборку программы Busybox следующими командами:

```
$ make defconfig
```

```

HOSTCC  scripts/basic/fixdep
HOSTCC  scripts/basic/split-include
HOSTCC  scripts/basic/docproc
GEN     include/applets.h
GEN     include/usage.h
GEN     klibc-utils/Kbuild
GEN     klibc-utils/Config.in
GEN     printutils/Kbuild
GEN     printutils/Config.in
GEN     archival/Kbuild
GEN     archival/Config.in
GEN     archival/libarchive/Kbuild
GEN     libbb/Kbuild
GEN     libbb/Config.in
GEN     coreutils/Kbuild
GEN     coreutils/Config.in
GEN     coreutils/libcoreutils/Kbuild
GEN     editors/Kbuild
GEN     editors/Config.in
GEN     procps/Kbuild
GEN     procps/Config.in
GEN     shell/Kbuild
GEN     shell/Config.in

```

Убедитесь, что сборка вашей программы тоже выполнена.

```

guz@Guzovski:~/busybox-src/busybox-1.36.1$ ls -l myprog libmystatic.a libmyshare
d.so
-rwxr-xr-x 1 guz guz 15984 Dec 12 19:20 libmyshared.so
-rw-r--r-- 1 guz guz 1710 Dec 12 19:20 libmystatic.a
-rwxr-xr-x 1 guz guz 16672 Dec 12 19:20 myprog

```

КОНТРОЛЬНЫЕ ВОПРОСЫ

- 1 Система сборки представляет собой инструмент автоматизации компиляции и связывания исходного кода она отслеживает зависимости между файлами перекомпилирует только изменённые компоненты упрощает сборку больших проектов и поддерживает установку и кросс-компиляцию
- 2 Make берёт исходные данные из файла Makefile в котором описаны цели зависимости и команды формат записи выглядит как цель двоеточие зависимости и команды с табуляцией сборка запускается командой make или make цель — это результат сборки например исполняемый файл зависимости — это файлы от которых она зависит абстрактная цель — это действие не связанное с файлом например clean или install
- 3 Make работает по алгоритму построения графа зависимостей проверяет временные метки файлов и выполняет команды только для тех целей которые устарели или зависят от изменённых файлов
- 4 Основные стандартные цели в Make это all — основная сборка проекта clean — удаление временных и объектных файлов install — установка собранных

файлов в целевой каталог `run` — запуск программы `test` — выполнение тестов

- 5 Переменные в Make позволяют задавать параметры сборки например компилятор `CC` или флаги `CFLAGS` их можно переопределять при запуске ограничения заключаются в том что имена не должны содержать пробелов значения представляют собой текстовые строки и в командах нельзя использовать пробелы вместо табуляции
- 6 Шаблонные правила создаются через конструкции вида `%.o: %.c` специальные переменные для них это `$@` — имя цели `$<` — первая зависимость `^` — все зависимости
- 7 Основные недостатки Make заключаются в сложности при масштабировании отсутствии встроенной конфигурации и чувствительности к форматированию альтернативные системы сборки включают CMake — кроссплатформенный инструмент который генерирует Makefile или Ninja Ninja — быстрый и минималистичный Meson — современный с простым синтаксисом и SCons — основанный на Python и гибкий в использовании

1. Как в утилите Make создаются шаблонные правила? Какие существуют специальные переменные для создания шаблонных правил?
2. В чем состоят основные недостатки системы сборки Make? Какие существуют альтернативные системы сборки и в чем их особенности?